

# Client Workflow Automation Agent — Technical Spec

**Purpose:** Automate three workflows currently done by hand in Karbon: (1) client follow-up chasing and rescheduling, (2) work progression triggered by external events (engagement signed, docs received), and (3) distinguishing "documents received" from "actually ready to start."

**Human-in-the-loop principle:** No client-facing communication is sent without a human able to review it first. This is achieved using Karbon's existing **Draft** and **Scheduled Send** email features — the agent fills in a draft from a template and schedules it, the preparer reviews/edits before it goes out.

**Architecture principle:** The system is split into two layers so that an upcoming workflow/tech-stack redesign only requires updating one of them.

- **Layer 1 — Core Toolkit:** reusable, workflow-agnostic building blocks that talk to Karbon, Outlook, and Claude. Build once. Should not need to change when your workflows are redesigned.
- **Layer 2 — Workflow Config:** the rules that say "when X happens, do Y using the toolkit." This is what encodes your current (soon to be revamped) process, and is what gets edited/rebuilt when the redesign lands.

This split means: build Layer 1 now if you want, since it doesn't depend on how your workflows end up configured. Hold off finalizing Layer 2 until the workflow redesign is done — or write it against your *current* process now, knowing it will need edits (not a rebuild) once the redesign lands.

## 1. Systems and Access

| System              | Role                                                       | Auth                                                                                                |
|---------------------|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Karbon API          | Work items, tasks, client requests, email drafts, webhooks | <code>Authorization</code> + <code>AccessKey</code> headers (per-user API key)                      |
| Ignition            | Engagement letter e-signature events                       | Native webhook ( <code>proposal.accepted</code> ) or Zapier "Proposal Accepted" trigger as fallback |
| Microsoft Graph API | Outlook calendar scheduling for preparers                  | OAuth2, <code>Calendars.ReadWrite</code> scope                                                      |

| System     | Role                                                   | Auth             |
|------------|--------------------------------------------------------|------------------|
| Claude API | Document completeness review, drafting fill-in content | Standard API key |

**Rate limit note:** Karbon API is capped at 120 requests/minute. Use webhooks instead of polling wherever Karbon supports them (work item changes, custom field changes) to stay well under this.

## 2. Layer 1 — Core Toolkit (build this regardless of workflow redesign)

These are generic functions/modules. None of them reference specific status names, work types, or checklists — they just perform an action given parameters.

### 2.1 Karbon connectors

- `getWorkItem(id)` / `updateWorkItemStatus(id, status)` / `updateWorkItemDate(id, newDate)`
- `getWorkItemAttachments(id)` — pull documents attached to a work item
- `createOrScheduleDraftEmail(templateId, recipient, fillValues, sendAt)` — populates an existing Karbon draft template and schedules it (never sends directly)
- `subscribeWebhook(eventType, url)` — for work item / custom field change events
- Auth wrapper handling the `AccessKey` header and rate-limit backoff (429 + `Retry-After`)

### 2.2 Outlook / Graph connector

- `createCalendarBlock(preparerId, start, end, subject)`
- `moveCalendarBlock(eventId, newStart, newEnd)`
- `deleteCalendarBlock(eventId)`
- OAuth2 token refresh handling

### 2.3 Ignition connector

- Webhook receiver for `proposal.accepted` (or Zapier "Proposal Accepted" trigger as a no-code fallback if native webhooks aren't wired up)
- `matchClientToKarbonContact(ignitionClientPayload)` — resolves an Ignition client to a Karbon contact/organization (needs a stable shared identifier — see Open Questions)

## 2.4 Claude connectors (generic, reusable across any checklist)

- `checkDocumentCompleteness(documents, checklist)` → returns present/missing items  
— the checklist is passed in as data, not hardcoded, so this function is identical no matter how your workflows change
- `draftFillValues(templateContext, clientInfo, missingItems)` → generates the specific text to drop into a Karbon draft's placeholders

## 2.5 Scheduling/timer engine

- `checkOverdueResponses(thresholdDays)` — generic cron job: for any work item awaiting client input, has more than N days passed with no logged response? N and "what counts as a response" are passed in as config, not hardcoded.

---

## 3. Layer 2 — Workflow Config (rebuild this once your process redesign lands)

This layer is a set of rules/data files, not core logic. It's intentionally kept thin so it's fast to edit.

### 3.1 Status/trigger mapping

A config table: `{ karbonStatusName, triggerCondition, toolkitActionsToRun }`. Example shape (values are placeholders — replace once your new statuses exist):

```
json
{
  "trigger": "documents_received",
  "condition": "attachment_added AND workItemType == '1040'",
  "actions": ["checkDocumentCompleteness", "branchOnResult"]
}
```

### 3.2 Checklists per return type

Data files, one per return type (1040, 1120S, 1065, etc.), each a list of explicit checkable items. **This is the piece most worth drafting now**, independent of the tech-stack redesign — it's about what documents you need, not how Karbon is configured. See Section 5.

### 3.3 Template ID mapping

Which Karbon draft template corresponds to which situation (missing-doc request, response reminder, ready-to-start notice, etc.) — will need to be re-pointed if templates get renamed/rebuilt in the redesign.

### 3.4 Response-timeout policy

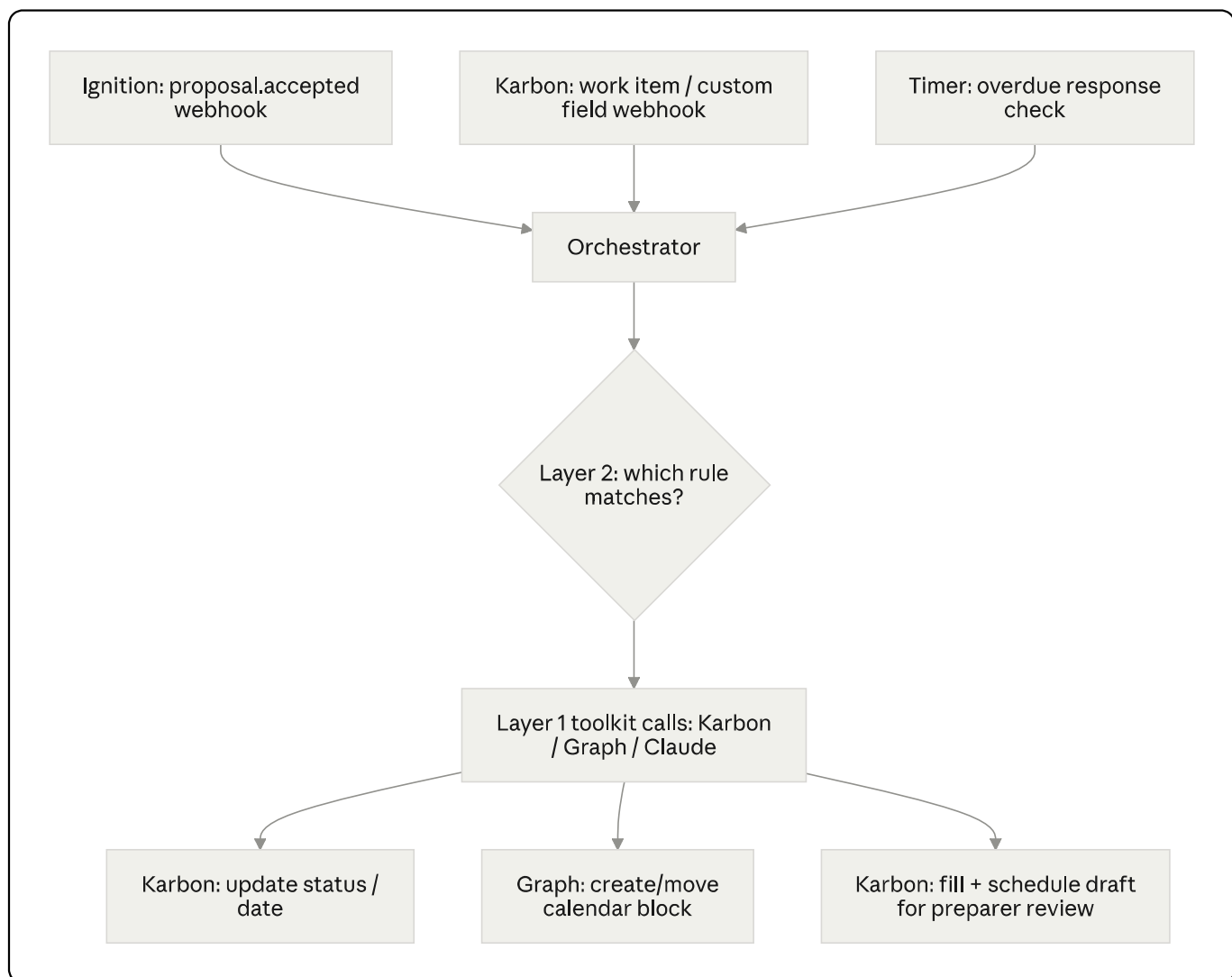
Number of business days before a follow-up fires, whether it changes near deadlines, and whose holiday calendar governs. Worth deciding now (Section 5) — it's a policy choice, not a Karbon config detail.

### 3.5 Work-type-specific scheduling rules

How far in advance to block calendar time once "Ready to Start" fires, whether it varies by return complexity, etc.

---

## 4. Architecture Diagram



The orchestrator itself is thin: it receives events, looks up the matching rule in Layer 2 config, and calls the relevant Layer 1 functions. Swapping Layer 2 config after the redesign shouldn't require touching the orchestrator or Layer 1 code.

---

## 5. Work Now, Independent of the Redesign

These don't depend on Karbon configuration and are worth doing in parallel:

1. **Checklists per return type** — explicit, checkable items (not "organizer complete" but "all W-2/1099 counts match organizer, K-1s received for all listed entities," etc.), plus what's commonly missing for your specific client base.
2. **Per-item missing-document handling** — does every gap get the same follow-up, or does urgency/wording vary by document type?
3. **Response-timeout policy** — business days threshold, deadline-proximity adjustments, holiday calendar.
4. **Client-identity mapping strategy** — how Ignition clients resolve to Karbon contacts (likely email address, but confirm).
5. **Layer 1 toolkit build** — your developer can start on the Karbon/Graph/Claude connector functions now, since none of them hardcode workflow-specific logic.

## 6. Hold Until Workflows Are Finalized

1. Layer 2 status/trigger mapping (depends on final Karbon status names and work types)
  2. Template ID mapping (depends on final draft templates)
  3. Any assumption about which system fires which webhook, if the tech-stack revamp changes tools beyond Karbon itself
- 

## 7. Open Questions

1. **Client identity mapping** — shared identifier between Ignition and Karbon?
  2. **Document-received signal** — does Karbon fire a webhook on attachment upload, or does this need another detection method (Triage activity, manual checklist tick)?
  3. **"Business days" calendar** — firm's calendar, or account for client time zones?
  4. **Error handling** — default to "flag for human review" when Claude's completeness check is ambiguous (e.g., illegible document) rather than guessing.
  5. **Scope of the tech-stack revamp** — is Karbon itself staying, or could it be replaced? If replaced, Layer 1's Karbon connector is also provisional.
-

## 8. Suggested Build Order

**Now:** Layer 1 toolkit (all connectors) + checklists + response-timeout policy — none of this is wasted regardless of how the redesign lands.

**After workflows finalize:** Layer 2 config — status mappings, template IDs, trigger rules. This is deliberately the fast part, since Layer 1 will already exist.

---

## 9. Additional Candidates (not in original scope — for later consideration)

These surfaced from looking at common pain points across firms using Karbon and similar practice management tools, not from the original three workflows. None are required for the MVP, but each reuses the same Layer 1 toolkit (particularly the time-in-status timer/escalation function), so they're relatively cheap to add later.

**Review/QC bottleneck escalation.** Reviews commonly stall when one senior preparer is the sole reviewer and gets pulled into other work, or is unexpectedly out with no easy way for others to see their queue. Candidate: flag work items sitting in "In Review" status past a set threshold, or surface a designated backup reviewer's view automatically when the primary is marked out. Uses the same `checkOverdueResponses`-style timer as Workflow C, just watching a different status.

**Extension risk flagging.** Karbon has native automated extension tracking; worth evaluating whether it covers what's needed before building a custom version. If not, a "days until deadline vs. current work item status" check is a natural extension of the same escalation logic used elsewhere — flag anything at risk of missing a filing deadline given its current stage.

**Billing trigger on completion.** If a work item's status change to "Complete" or "Approved" is already being watched for scheduling purposes, the same event can trigger invoice creation (either directly via Karbon's billing feature or via Ignition, depending on which system owns invoicing in the redesigned stack). Small addition, real payoff, no new toolkit component required.

**Post-completion client loop.** Once a job closes, an automated e-file authorization (Form 8879) request or satisfaction survey can go out — mechanically the same "fill a draft template, schedule for review" action used for organizer requests and follow-ups, just triggered at the other end of the pipeline.

**Data handling for client documents.** Not a feature — a design constraint that should be settled before Workflow B is built. Since the completeness-check step has Claude reading source documents that likely contain SSNs and financial account numbers, decisions are needed on: retention (documents shouldn't persist in the orchestrator or logs longer than

needed to complete the check), access logging, and encryption in transit/at rest. This should be reviewed against the firm's existing WISP (Written Information Security Plan) and Circular 230 obligations — the same standards that already govern how these documents are handled manually.